

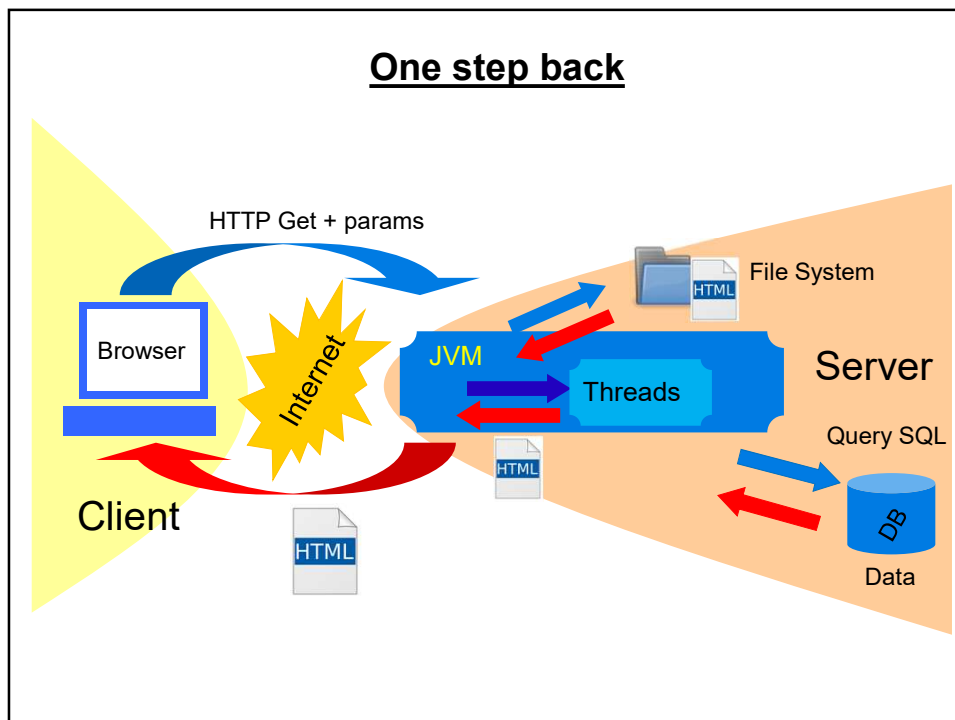


UNIVERSITÀ
DI TRENTO

Access to DB

1

One step back



DB and DBMS

- A database (DB) is a collection of data with a correlated meaning that are stored and managed in a computer system
- A Database Management System (DBMS) is a software application conceived to store, retrieve, manage data in DBs

We will often mix up the terms DB and DBMS
(for our purposes we should always use DBMS!)

3

Relational DBs

Most of today's DBs are relational DBs, that is they are based on the relational data model. The key components of a relational data model are:

- **Structure:** defining how data are represented
- **Integrity:** putting constraints on data
- **Language:** providing ways for accessing / manipulating data

4

Few words about structure

- A relational DB consists of a set of relations
- A relation is a set of tuples whose items are elements of a set
- A relation has:
 - **a schema**: defining the relation (define the sets to which the elements belong)

`Student (ID: integer; Name: string; Country: string)`

- **an instance** of the schema: being the content of the relation at a given time. Basically, an instance is a table with rows and columns

279089	Alice	IT
265680	Bob	UK
299934	Tom	FR

5

Few words about integrity

- There are two classes of constraints:
 - **Intra-relational constraints**, that is constraints involving just one relation. They can be:
 - Domain constraints
 - Primary key constraints
 - **Inter-relational constraints**, that is constraints involving more than one relation. They are:
 - Foreign key constraints

6

Summary on constraints

Enrollment table

Stud. ID	Course ID	Date	Mark
2288432	1111	2024/01/19	A
2288432	1122	2024/02/16	A
2288432	1102	2024/02/26	C

Course table

Course ID	Topic ID	Course title	ECTS
1111	CS	OOP Programming: Java	6
1122	CS	Introduction to web programming	6
1102	MATH	Calculus and linear algebra	9

Each value in Course ID in the Enrollment table MUST match a value in Course ID in the Course table

Each row MUST have a Course ID, and its MUST be unique

Each value in ECTS MUST be greater than 0 and less than 12

7

Few words about language

- The **Structured Query Language (SQL)** is the universal language for accessing relational DBs
- Applications may allow users to access DBs without directly using SQL. These applications, however, must use SQL to access DBs

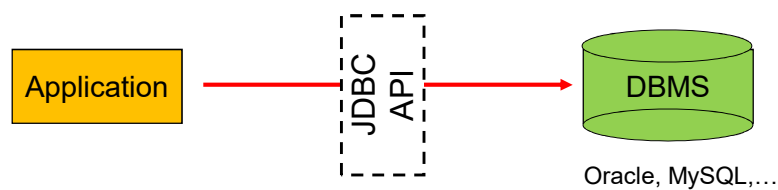
<https://www.w3schools.com/sql/>

8

How do I access a database from Java?

9

Java DataBase Connectivity (JDBC)

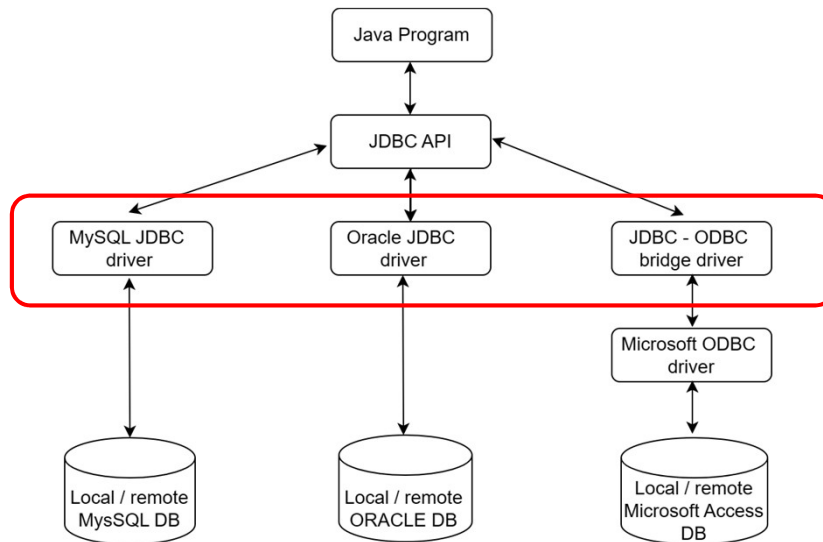


- JDBC is an interface which allows Java code to:
 - Establish a connection with a DBMS
 - Send SQL statements
 - Process results

How JDBC can "talk" with a so large number of different technologies?

10

The architecture of JDBC



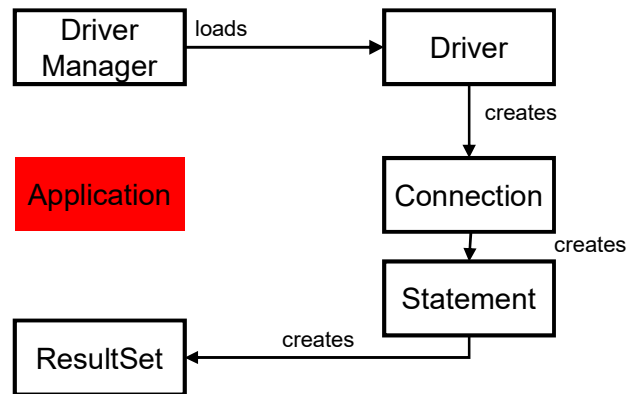
11

The JDBC driver

- The driver:
 - Implements the JDBC API interfaces
 - Executes the methods invoked by the Java application
 - Translates these operations into commands specific to the DBMS
 - Manages communication with the DBMS
 - Receives results from the DBMS and returns them to the application

12

The java.sql Object Model



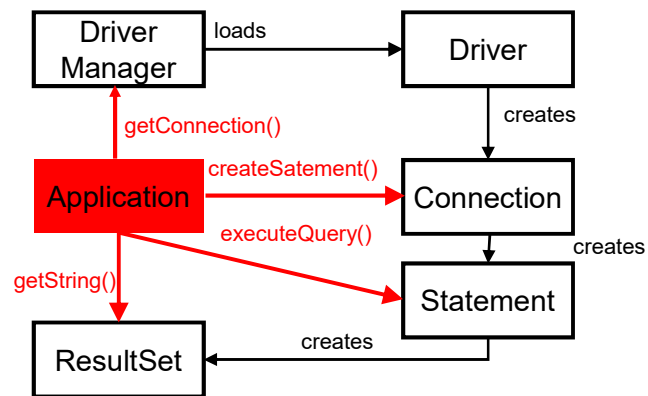
13

The java.sql Object Model

- **Driver Manager:**
 - it manages the registered JDBC drivers and choose the suitable one
- **Connection:**
 - It represents an active connection between the application and the DBMS
- **Statement:**
 - It represents an SQL command to be executed on the DBMS
- **ResultSet:**
 - It represents the set of results returned by an SQL query

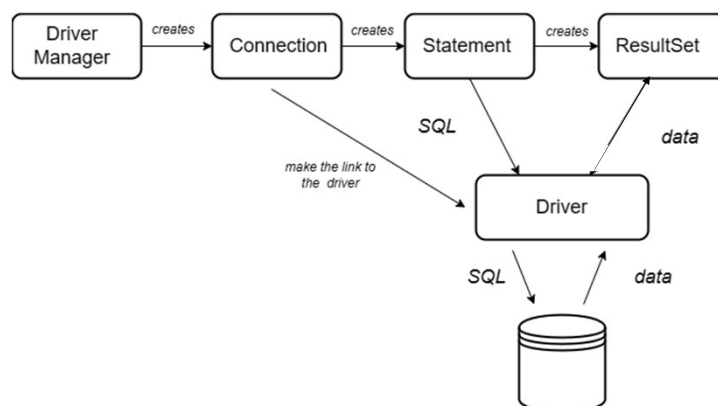
14

The java.sql Object Model



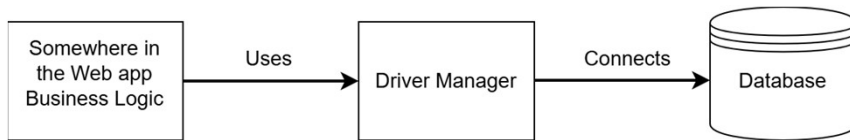
15

How JDBC objects interact with a DB



16

Let's think about it!



- When the web app accesses the DriverManager directly, it gets a new connection every time it needs one
- So, a DB connection could be opened for each request or for each session...

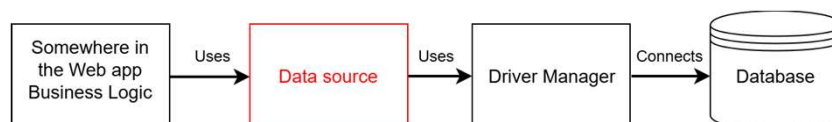
Getting connections often results in a lot of traffic over the network and multiple unnecessary authentications to the DB

Remember: DB connections are costly to create in terms of both response time and system resources

17

The Spring's solution

- Spring uses a **data source** object to efficiently retrieve and manage the connections to minimize the number of unnecessary operations
- A data source is a component that manages connections:
 - It provides the web app with connections when it's requested;
 - It creates new connections only when it is necessary;
 - It closes connections when they are not more needed.



Amazing, but how it does it?

18

Data Source

- It is a Spring bean (good news!)
- Spring Boot provides auto-configuration (hidden) for Data Source:

```
spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=
spring.datasource.platform=h2
```

- Developers can override all or part of such a config through the application properties file

19

Connection pooling

- Connection pooling is a strategy to manage and optimize the usage of connections to a DB in a web app
- How does it work?
 - **Pool creation and initialization:** at the starting of the web app, the connection pool is created and initialized based on the configuration provided. A few idle connections are pre-created. An idle connection is a connection ready to be used but not in use.
 - **Connection allocation:** when the web app needs a connection, the pool checks if any idle connections are available: if yes, it's allocated to the application. If not, and the maximum number supported by the pool is not reached yet, a new connection is created and allocated. If all the connections supported by the pool are in use, the pool waits for a connection to become available.
 - **Connection Reuse:** After the application finishes using the connection, it returns it to the pool and it is re-used. Connections are closed automatically after a certain period of idle time.

20

HikariCP

- It is the default connection pooling used in Spring Boot
- Key features: Fast, Minimalistic, Thread-safe, Efficient, Open source

More info at: <https://github.com/brettwooldridge/HikariCP>

```
# HikariCP connection pool settings
spring.datasource.hikari.maximum-pool-size=10 # Max connections in the pool
spring.datasource.hikari.minimum-idle=5      # Minimum idle connections
spring.datasource.hikari.idle-timeout=600000  # Max time a connection can be idle
spring.datasource.hikari.connection-timeout=30000 # Max time to wait for a connection
spring.datasource.hikari.max-lifetime=1800000 # Max lifetime of a connection in the pool
```

21

Pros in using a data source

- Using a connection pooling strategy to minimize the overhead of opening and closing connections
- Having a simplified management of database connections
- Decoupling the connection management mechanism and the used DB (you do not need to change much of the application code)

22

Working with persisted data

- Using JDBC classes provided by JDK, you have to write verbose blocks of code to work on data
- Spring work on data through:
 - beans annotated with `@Repository`;
 - a tool (very useful for small web app) named `JdbcTemplate`



A Repository is a class in charge to work with a DB; together with JDBC Template it is a Java's abstraction of DBMS

23

@Repository Bean and JDBC Template

- `@Repository` beans:
 - Indicate Spring that they are a data-access component
 - Organize methods to access data
 - Use JDBC Template
- JDBC Template:
 - It is a Spring helper class provided as a bean
 - Spring Boot will create it automatically if a `DataSource` exists
 - Executes SQL queries and statements
 - Manages DBMS connections (obtained from the `DataSource`)
 - Maps query results to objects

24

A one-sentence summary

A request arrives at a Controller, which eventually calls a Service; the Controller/Service calls the Repository, which uses JdbcTemplate; JdbcTemplate obtains a connection from the DataSource, executes the query on the DBMS, and the result is returned back up the chain

25

Our DB: H2

- H2 is a relational DBMS written in Java
- It can work in-memory, embedded, or in client-server mode
- Spring Boot **automatically** configure H2 in in-memory mode when it detects its dependency in your project

The screenshot shows the H2 Database Engine website. At the top, there's a blue header with the H2 logo and the text 'H2 Database Engine'. Below the header, there's a 'Welcome to H2, the Java SQL database. The main features of H2 are:' section followed by a list of features: 'Very fast, open source, JDBC API', 'Embedded and server modes; in-memory databases', 'Browser based Console application', and 'Small footprint: around 2.5 MB jar file size'. To the left of the main content, there's a sidebar with navigation links: 'Home', 'Download', 'Cheat Sheet', 'Documentation', 'Quickstart', 'Installation', 'Tutorial', 'Features', 'Security', 'Performance', 'Advanced', 'Reference', 'Commands', 'Functions', 'Data Types', 'SQL Grammar', 'System Tables', and 'Javadoc'. There's also a 'Search:' bar. On the right side, there's a 'Download' section with links for 'Version 2.3.232 (2024-08-11)', 'Windows Installer (6.7 MB)', 'All Platforms (zip, 9.5 MB)', and 'All Downloads'. Below that is a 'Support' section with links for 'Stack Overflow (tag H2)', 'Google Group', and a note about non-technical issues. At the bottom, there's a 'Features' section with a list of features: 'Very fast, open source, JDBC API', 'Embedded and server modes; disk-based or in-memory databases', 'Transaction support, multi-version concurrency', 'Browser based Console application', 'Encrypted databases', 'Fulltext search', and 'Pure Java with small footprint: around 2.5 MB jar file size'.

26

The dependencies you need

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-jdbc</artifactId>
</dependency>

<dependency>
  <groupId>com.h2database</groupId>
  <artifactId>h2</artifactId>
  <scope>runtime</scope>
</dependency>
```

pom.xml

- Inclusion of Spring JDBC and automatic configuration of a DataSource and of a JDBC Template
- Inclusion of the H2 driver working in in-memory mode

27

In-memory, embedded, client-server mode

Mode	Where H2 runs	Location	When to use it
In-memory	Same JVM as your web app	RAM	Prototyping, testing
Embedded	Same JVM as your web app	Local file	Lightweight web apps, no need for an external DB
Client-server	A separate server process	Local / Remote file	Realistic production-like setup

28

Setting up tables and data

- In `src/main/resources`:
 - Add a `schema.sql` file in (schema defines tables)
 - If you want having initial data, add a file `data.sql` with it
- When the web app starts:
 - Spring Boot:
 - Read `schema.sql` and creates tables
 - Read data and insert it
 - Repositories and JDBC Template can directly work
- The file names are those ones used by Spring Boot. You can change them in the application properties file

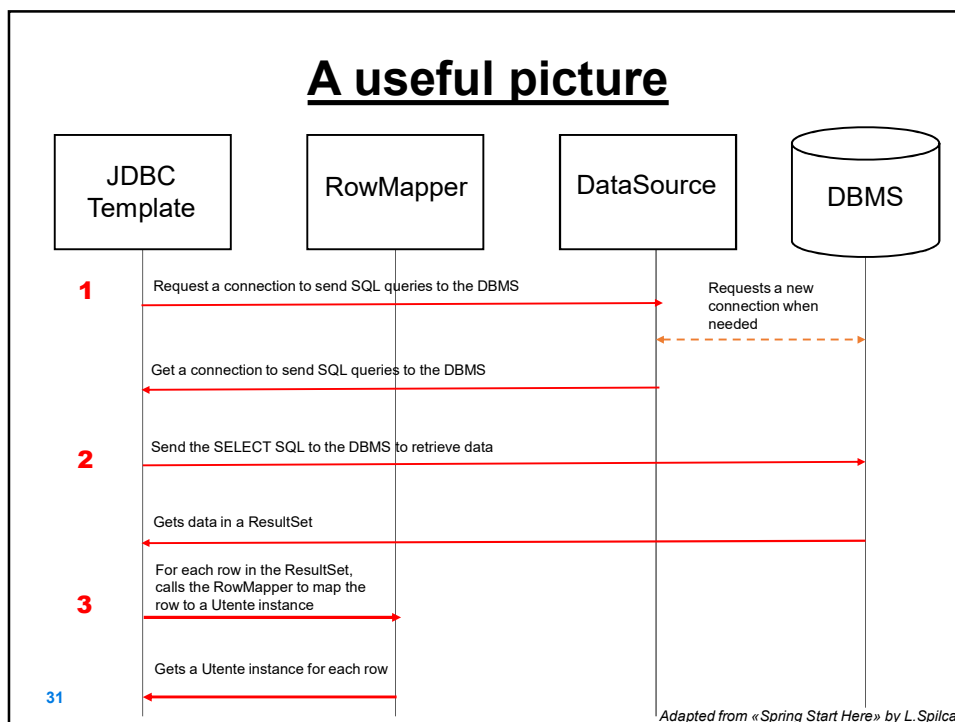
29

Let's play with our DB

`esempio18`

30

A useful picture



Something new about thymeleaf

- In Thymeleaf, we can implement iteration using the **th:each** attribute
- **th:each** accept and iterate over some different data types, such as:
 - objects implementing *java.util.Iterable*
 - objects implementing *java.util.Map*
 - arrays
 - any other object is treated as if it were a single-valued list containing one element

Example

```
<tr th:each="user: ${users}"> <td th:text="${user.id}" /> <td
th:text="${user.name}" /> </tr>
```

The code shows **th:each** iterating over a list of *users*. Each element of the list is passed to the body of the loop via the *user* variable.

Prepared Statements

- **Prepared Statements** are precompiled SQL statements that can be executed multiple times efficiently, with different parameters. So, they are parametric SQL statements

```
String sql = "INSERT INTO coffee (brand, price) VALUES (?,?)";
```

The «?» are placeholders of the parameters that will be bound dynamically at runtime

So, prepared statements separates SQL statement logic from the data being used in the statements

33

Why prepared statements?

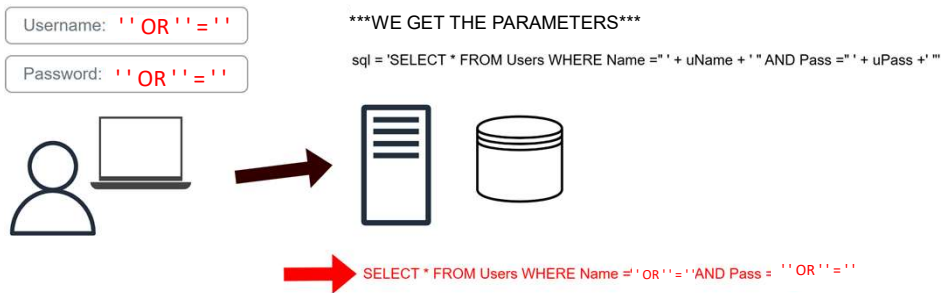
- They can be reused multiple times with different parameters, reducing the need to re-compile the statement, thus improving performance for frequently executed statements
- They prevent SQL injection



34

SQL Injection

- SQL injection is a very common web hacking strategy



35

SQL injection & prepared statements

- Using prepared statements, the SQL engine binds the values to the placeholder (?) before executing the statement
- So, the values are treated as data and not as a part of SQL syntax

36

Let's play again with our DB

esempio19

37